

Disjoint Set Union

Sean

August 25, 2026

Outline

- Merge-only components
- Parent forest representation
- Making DSU fast
- Compact one-array DSU
- Component metadata
- Cycle detection
- DSU vs DFS/BFS
- Rollback DSU
- Problem patterns



Why DSU?

- Many problems ask whether two objects are already connected

Why DSU?

- Many problems ask whether two objects are already connected
- Then an update adds a new connection

Why DSU?

- Many problems ask whether two objects are already connected
- Then an update adds a new connection
- Connections only merge components; they never delete edges

Why DSU?

- Many problems ask whether two objects are already connected
- Then an update adds a new connection
- Connections only merge components; they never delete edges
- DSU gives a simple way to maintain those changing components



- 1 Merge-only components
- 2 Parent forest representation
- 3 Making DSU fast
- 4 Compact one-array DSU
- 5 Component metadata
- 6 Cycle detection
- 7 DSU vs DFS/BFS
- 8 Rollback DSU
- 9 Problem patterns

Merge-only components

The DSU model

- Start with n separate sets: $\{1\}, \{2\}, \dots, \{n\}$

The DSU model

- Start with n separate sets: $\{1\}, \{2\}, \dots, \{n\}$
- `find(x)` returns the representative of x 's set



The DSU model

- Start with n separate sets: $\{1\}, \{2\}, \dots, \{n\}$
- `find(x)` returns the representative of x 's set
- `unite(a, b)` merges the sets containing a and b

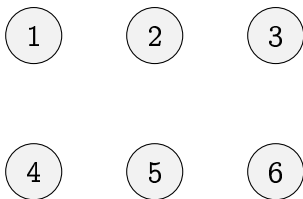


The DSU model

- Start with n separate sets: $\{1\}, \{2\}, \dots, \{n\}$
- `find(x)` returns the representative of x 's set
- `unite(a, b)` merges the sets containing a and b
- `same(a, b)` checks whether two elements have the same representative

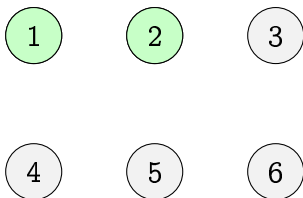


Component trace



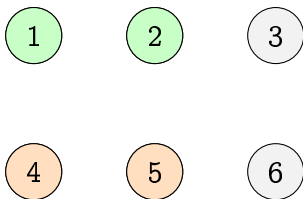
- Start separate

Component trace



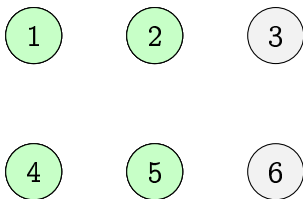
- Start separate
- `unite(1, 2)`

Component trace



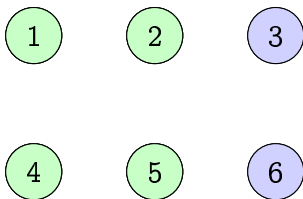
- Start separate
- `unite(1, 2)`
- `unite(4, 5)`

Component trace



- Start separate
- `unite(1, 2)`
- `unite(4, 5)`
- `unite(2, 5)`

Component trace



- Start separate
- `unite(1, 2)`
- `unite(4, 5)`
- `unite(2, 5)`
- `unite(3, 6)`

When DSU fits

- Good: add an edge, ask whether two vertices are connected

When DSU fits

- Good: add an edge, ask whether two vertices are connected
- Good: group items by equality constraints

When DSU fits

- Good: add an edge, ask whether two vertices are connected
- Good: group items by equality constraints
- Good: build components while processing events offline or in sorted order

When DSU fits

- Good: add an edge, ask whether two vertices are connected
- Good: group items by equality constraints
- Good: build components while processing events offline or in sorted order
- Not enough by itself: delete an edge and answer online connectivity



- 1 Merge-only components
- 2 Parent forest representation**
- 3 Making DSU fast
- 4 Compact one-array DSU
- 5 Component metadata
- 6 Cycle detection
- 7 DSU vs DFS/BFS
- 8 Rollback DSU
- 9 Problem patterns

Parent forest representation

Representatives

- Each set chooses one element as its representative

Representatives

- Each set chooses one element as its representative
- Two elements are in the same set exactly when their representatives match

Representatives

- Each set chooses one element as its representative
- Two elements are in the same set exactly when their representatives match
- DSU stores a parent pointer for every element

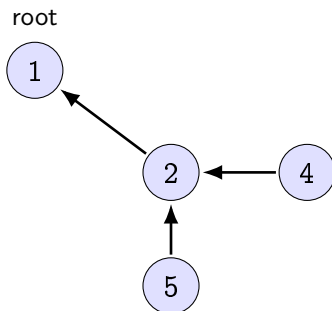


Representatives

- Each set chooses one element as its representative
- Two elements are in the same set exactly when their representatives match
- DSU stores a parent pointer for every element
- A representative is a root: $\text{parent}[x] == x$

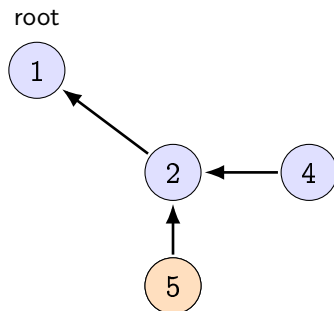


Finding the root



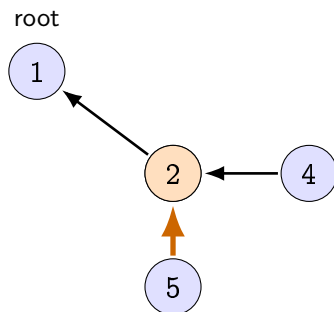
- Parent pointers form trees

Finding the root



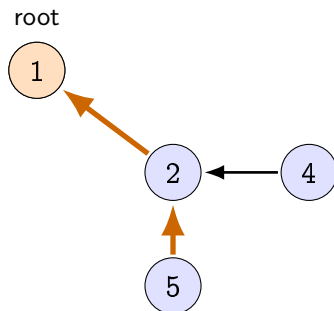
- Parent pointers form trees
- To compute `find(5)`, start at 5

Finding the root



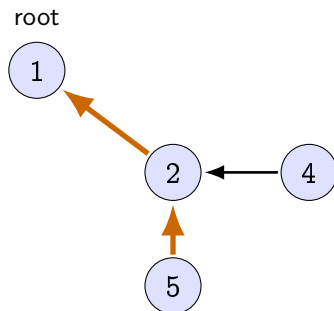
- Parent pointers form trees
- To compute `find(5)`, start at 5
- Move to parent[5] = 2

Finding the root



- Parent pointers form trees
- To compute `find(5)`, start at 5
- Move to parent[5] = 2
- Move to parent[2] = 1

Finding the root



- Parent pointers form trees
- To compute `find(5)`, start at 5
- Move to parent[5] = 2
- Move to parent[2] = 1
- Representative is 1

Merging two sets

- First find the roots: $ra = \text{find}(a)$, $rb = \text{find}(b)$



Merging two sets

- First find the roots: $ra = \text{find}(a)$, $rb = \text{find}(b)$
- If $ra = rb$, the elements are already in the same set



Merging two sets

- First find the roots: $ra = \text{find}(a)$, $rb = \text{find}(b)$
- If $ra = rb$, the elements are already in the same set
- Otherwise, make one root point to the other root

Merging two sets

- First find the roots: $ra = \text{find}(a)$, $rb = \text{find}(b)$
- If $ra = rb$, the elements are already in the same set
- Otherwise, make one root point to the other root
- Only roots should be attached during `unite`



First implementation

```
1  struct DSU {
2      vector<int> parent;
3
4      DSU(int n) : parent(n + 1) {
5          iota(parent.begin(), parent.end(), 0);
6      }
7
8      int find(int x) {
9          while (parent[x] != x) x = parent[x];
10         return x;
11     }
12
13     bool unite(int a, int b) {
14         a = find(a);
15         b = find(b);
16         if (a == b) return false;
17         parent[b] = a;
18         return true;
19     }
20 };
```



- 1 Merge-only components
- 2 Parent forest representation
- 3 **Making DSU fast**
- 4 Compact one-array DSU
- 5 Component metadata
- 6 Cycle detection
- 7 DSU vs DFS/BFS
- 8 Rollback DSU
- 9 Problem patterns



Making DSU fast

The problem with naive merging

- If we attach roots carelessly, the tree can become a long chain

The problem with naive merging

- If we attach roots carelessly, the tree can become a long chain
- Then `find(x)` may walk through many parents



The problem with naive merging

- If we attach roots carelessly, the tree can become a long chain
- Then `find(x)` may walk through many parents
- A bad sequence can make one operation cost $\mathcal{O}(n)$

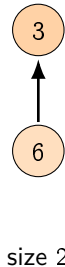
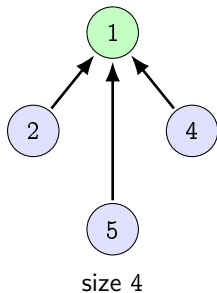


The problem with naive merging

- If we attach roots carelessly, the tree can become a long chain
- Then `find(x)` may walk through many parents
- A bad sequence can make one operation cost $\mathcal{O}(n)$
- DSU uses two small ideas to avoid this

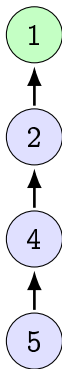


Union by size



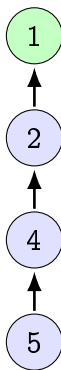
Attach the smaller tree under the larger tree.

Path compression



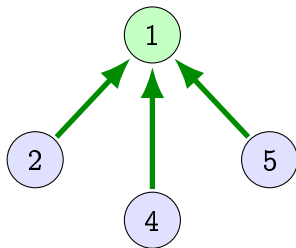
- `find(5)` follows the path to root 1

Path compression



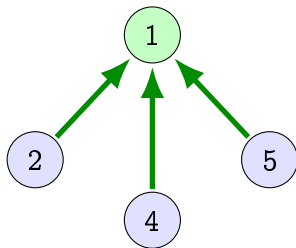
- `find(5)` follows the path to root 1
- While returning, remember that root

Path compression



- `find(5)` follows the path to root 1
- While returning, remember that root
- Point every visited node directly at 1

Path compression



- `find(5)` follows the path to root 1
- While returning, remember that root
- Point every visited node directly at 1
- Future finds are much shorter

Full DSU code

```
1  struct DSU {
2      vector<int> parent, sz;
3
4      DSU(int n) : parent(n + 1), sz(n + 1, 1) {
5          iota(parent.begin(), parent.end(), 0);
6      }
7
8      int find(int x) {
9          if (parent[x] == x) return x;
10         return parent[x] = find(parent[x]);
11     }
12
13     bool unite(int a, int b) {
14         a = find(a);
15         b = find(b);
16         if (a == b) return false;
17         if (sz[a] < sz[b]) swap(a, b);
18         parent[b] = a;
19         sz[a] += sz[b];
20         return true;
21     }
22 };
```



Common helpers

```
1  bool same(int a, int b) {  
2      return find(a) == find(b);  
3  }  
4  
5  int size(int x) {  
6      return sz[find(x)];  
7  }
```

- Keep `sz[root]` meaningful only at representatives
- Call `find(x)` before reading component information



Complexity

- With both optimizations, DSU operations are nearly constant time



Complexity

- With both optimizations, DSU operations are nearly constant time
- The formal bound is $\mathcal{O}(\alpha(n))$ amortized per operation



Complexity

- With both optimizations, DSU operations are nearly constant time
- The formal bound is $\mathcal{O}(\alpha(n))$ amortized per operation
- $\alpha(n)$ is the inverse Ackermann function



Complexity

- With both optimizations, DSU operations are nearly constant time
- The formal bound is $\mathcal{O}(\alpha(n))$ amortized per operation
- $\alpha(n)$ is the inverse Ackermann function
- For competitive programming inputs, treat it as at most a tiny constant



How small is $\alpha(n)$?

- The Ackermann function grows much faster than exponentials

How small is $\alpha(n)$?

- The Ackermann function grows much faster than exponentials
- $\alpha(n)$ asks how many levels of that growth are needed to reach n

How small is $\alpha(n)$?

- The Ackermann function grows much faster than exponentials
- $\alpha(n)$ asks how many levels of that growth are needed to reach n
- Even a tower like $2^{2^{2^{65536}}}$ only moves the answer by a small constant



How small is $\alpha(n)$?

- The Ackermann function grows much faster than exponentials
- $\alpha(n)$ asks how many levels of that growth are needed to reach n
- Even a tower like $2^{2^{2^{65536}}}$ only moves the answer by a small constant
- For contest constraints, think of $\alpha(n)$ as at most 4 or 5



How small is $\alpha(n)$?

- The Ackermann function grows much faster than exponentials
- $\alpha(n)$ asks how many levels of that growth are needed to reach n
- Even a tower like $2^{2^{2^{65536}}}$ only moves the answer by a small constant
- For contest constraints, think of $\alpha(n)$ as at most 4 or 5
- The practical takeaway: DSU operations behave like constant time



- 1 Merge-only components
- 2 Parent forest representation
- 3 Making DSU fast
- 4 **Compact one-array DSU**
- 5 Component metadata
- 6 Cycle detection
- 7 DSU vs DFS/BFS
- 8 Rollback DSU
- 9 Problem patterns

Compact one-array DSU

Store parents and sizes together

- Another common DSU style uses one array, often called e



Store parents and sizes together

- Another common DSU style uses one array, often called e
- If x is a root, then $e[x]$ is negative



Store parents and sizes together

- Another common DSU style uses one array, often called e
- If x is a root, then $e[x]$ is negative
- The component size is $-e[x]$



Store parents and sizes together

- Another common DSU style uses one array, often called e
- If x is a root, then $e[x]$ is negative
- The component size is $-e[x]$
- If x is not a root, then $e[x]$ is the parent of x



Store parents and sizes together

- Another common DSU style uses one array, often called e
- If x is a root, then $e[x]$ is negative
- The component size is $-e[x]$
- If x is not a root, then $e[x]$ is the parent of x
- Initialize with $e.assign(n, -1)$: every vertex is a root of size 1



Compact DSU core

```
1  struct DSU {
2      vector<int> e;
3
4      DSU(int n) : e(n, -1) {}
5
6      int get(int x) {
7          return e[x] < 0 ? x : e[x] = get(e[x]);
8      }
9
10     bool same(int x, int y) {
11         return get(x) == get(y);
12     }
13
14     int size(int x) {
15         return -e[get(x)];
16     }
17 };
```



Union with negative sizes

```
1  bool unite(int x, int y) {
2      x = get(x);
3      y = get(y);
4      if (x == y) return false;
5
6      // More negative means larger component.
7      if (e[x] > e[y]) swap(x, y);
8
9      e[x] += e[y];
10     e[y] = x;
11     return true;
12 }
```

- Example: size 5 root stores -5, size 2 root stores -2
- Since $-5 < -2$, root x should keep the smaller array value



Metadata still works

```
1  vector<int> e, mn, mx;
2
3  DSU(int n) : e(n, -1), mn(n), mx(n) {
4      iota(mn.begin(), mn.end(), 0);
5      iota(mx.begin(), mx.end(), 0);
6  }
7
8  bool unite(int x, int y) {
9      x = get(x);
10     y = get(y);
11     if (x == y) return false;
12     if (e[x] > e[y]) swap(x, y);
13
14     e[x] += e[y];
15     e[y] = x;
16     mn[x] = min(mn[x], mn[y]);
17     mx[x] = max(mx[x], mx[y]);
18     return true;
19 }
```



Querying compact metadata

```
1  int Min(int x) {  
2      return mn[get(x)];  
3  }  
4  
5  int Max(int x) {  
6      return mx[get(x)];  
7  }
```

- Same rule as before: read metadata at the current root
- If vertex labels are 1 to n , initialize `mn` and `mx` from 1



- 1 Merge-only components
- 2 Parent forest representation
- 3 Making DSU fast
- 4 Compact one-array DSU
- 5 Component metadata
- 6 Cycle detection
- 7 DSU vs DFS/BFS
- 8 Rollback DSU
- 9 Problem patterns



Component metadata

Metadata lives at the root

- DSU can store information about each component, not just its representative



Metadata lives at the root

- DSU can store information about each component, not just its representative
- Examples: size, minimum vertex, maximum vertex, sum of values, number of edges



Metadata lives at the root

- DSU can store information about each component, not just its representative
- Examples: size, minimum vertex, maximum vertex, sum of values, number of edges
- Store the data in arrays indexed by the component root



Metadata lives at the root

- DSU can store information about each component, not just its representative
- Examples: size, minimum vertex, maximum vertex, sum of values, number of edges
- Store the data in arrays indexed by the component root
- Before reading metadata for x , use $r = \text{find}(x)$



Metadata lives at the root

- DSU can store information about each component, not just its representative
- Examples: size, minimum vertex, maximum vertex, sum of values, number of edges
- Store the data in arrays indexed by the component root
- Before reading metadata for x , use $r = \text{find}(x)$
- Data stored at non-roots may be stale after merges



Merging metadata

```
1  vector<int> parent, sz, mn, mx;
2  vector<long long> sum;
3
4  // Initially, root i stores the metadata for component {i}.
5  mn[i] = mx[i] = i;
6  sum[i] = value[i];
7
8  bool unite(int a, int b) {
9      a = find(a);
10     b = find(b);
11     if (a == b) return false;
12     if (sz[a] < sz[b]) swap(a, b);
13
14     parent[b] = a;
15     sz[a] += sz[b];
16     mn[a] = min(mn[a], mn[b]);
17     mx[a] = max(mx[a], mx[b]);
18     sum[a] += sum[b];
19     return true;
20 }
```



Querying metadata

```
1  int root = dsu.find(x);
2
3  cout << dsu.sz[root] << '\n';
4  cout << dsu.mn[root] << ' ' << dsu.mx[root] << '\n';
5  cout << dsu.sum[root] << '\n';
```

- Metadata queries are component queries
- Always find the current root first



Counting edges per component

```
1  bool add_edge(int a, int b) {
2      a = find(a);
3      b = find(b);
4      if (a == b) {
5          edges[a]++;
6          return false;
7      }
8      if (sz[a] < sz[b]) swap(a, b);
9      parent[b] = a;
10     sz[a] += sz[b];
11     edges[a] += edges[b] + 1;
12     return true;
13 }
```



What metadata can answer

- What is the size of x 's component?

What metadata can answer

- What is the size of x 's component?
- What is the minimum or maximum label in this component?

What metadata can answer

- What is the size of x 's component?
- What is the minimum or maximum label in this component?
- What is the sum of values in this component?

What metadata can answer

- What is the size of x 's component?
- What is the minimum or maximum label in this component?
- What is the sum of values in this component?
- Does this component contain a cycle? Check whether edge count is at least vertex count

- 1 Merge-only components
- 2 Parent forest representation
- 3 Making DSU fast
- 4 Compact one-array DSU
- 5 Component metadata
- 6 Cycle detection
- 7 DSU vs DFS/BFS
- 8 Rollback DSU
- 9 Problem patterns



Cycle detection

Undirected cycle rule

- Process edges one at a time

Undirected cycle rule

- Process edges one at a time
- Before adding (u, v) , check whether u and v are already connected



Undirected cycle rule

- Process edges one at a time
- Before adding (u, v) , check whether u and v are already connected
- If they are in different components, `unite`(u , v)

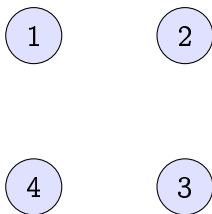


Undirected cycle rule

- Process edges one at a time
- Before adding (u, v) , check whether u and v are already connected
- If they are in different components, `unite`(u , v)
- If they are already in the same component, this edge closes a cycle

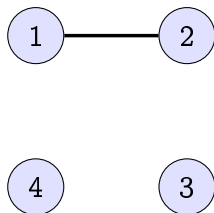


Cycle detection trace



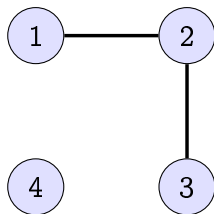
- Start with four separate vertices

Cycle detection trace



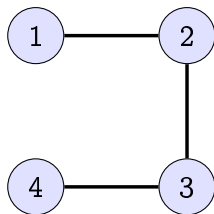
- Start with four separate vertices
- Add (1, 2)

Cycle detection trace



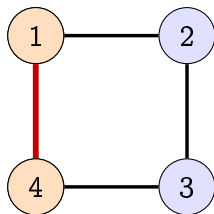
- Start with four separate vertices
- Add (1, 2)
- Add (2, 3)

Cycle detection trace



- Start with four separate vertices
- Add (1, 2)
- Add (2, 3)
- Add (3, 4)

Cycle detection trace



- Start with four separate vertices
- Add (1, 2)
- Add (2, 3)
- Add (3, 4)
- (4, 1) connects inside the same component

Cycle detection code

```
1 DSU dsu(n);
2 bool has_cycle = false;
3
4 for (auto [u, v] : edges) {
5     if (dsu.same(u, v)) {
6         has_cycle = true;
7         break;
8     }
9     dsu.unite(u, v);
10 }
```

- This detects whether an undirected graph has any cycle



Details to mention

- This rule is for undirected graphs

Details to mention

- This rule is for undirected graphs
- A self-loop is immediately a cycle

Details to mention

- This rule is for undirected graphs
- A self-loop is immediately a cycle
- A repeated parallel edge creates a cycle after the first copy

Details to mention

- This rule is for undirected graphs
- A self-loop is immediately a cycle
- A repeated parallel edge creates a cycle after the first copy
- DSU can detect that a cycle exists, but it does not recover the vertices on the cycle



Details to mention

- This rule is for undirected graphs
- A self-loop is immediately a cycle
- A repeated parallel edge creates a cycle after the first copy
- DSU can detect that a cycle exists, but it does not recover the vertices on the cycle
- Directed cycle detection needs a different tool, usually DFS colors



- 1 Merge-only components
- 2 Parent forest representation
- 3 Making DSU fast
- 4 Compact one-array DSU
- 5 Component metadata
- 6 Cycle detection
- 7 DSU vs DFS/BFS**
- 8 Rollback DSU
- 9 Problem patterns



DSU vs DFS/BFS

DSU vs graph search

- DFS/BFS answers connectivity by exploring the graph now



DSU vs graph search

- DFS/BFS answers connectivity by exploring the graph now
- DSU answers connectivity from maintained component representatives



DSU vs graph search

- DFS/BFS answers connectivity by exploring the graph now
- DSU answers connectivity from maintained component representatives
- DFS/BFS is great for one static graph traversal



DSU vs graph search

- DFS/BFS answers connectivity by exploring the graph now
- DSU answers connectivity from maintained component representatives
- DFS/BFS is great for one static graph traversal
- DSU is great for many edge additions and connectivity queries



DSU vs graph search

- DFS/BFS answers connectivity by exploring the graph now
- DSU answers connectivity from maintained component representatives
- DFS/BFS is great for one static graph traversal
- DSU is great for many edge additions and connectivity queries
- DSU does not keep the actual paths between vertices



When to use which?

Use DFS/BFS

Need traversal order, distances, paths, or connected components once

Use DSU

Edges are added over time and queries ask whether two nodes are connected

Use other tools

Online edge deletions, shortest paths, directed reachability

- If the graph is changing only by adding edges, DSU avoids repeated searches



Common mistake

- Suppose there are q operations on an n -node, m -edge graph

Common mistake

- Suppose there are q operations on an n -node, m -edge graph
- Running DFS/BFS for every connectivity query may cost $\mathcal{O}(q(n + m))$



Common mistake

- Suppose there are q operations on an n -node, m -edge graph
- Running DFS/BFS for every connectivity query may cost $\mathcal{O}(q(n + m))$
- DSU processes each edge addition or query in nearly constant time



Common mistake

- Suppose there are q operations on an n -node, m -edge graph
- Running DFS/BFS for every connectivity query may cost $\mathcal{O}(q(n + m))$
- DSU processes each edge addition or query in nearly constant time
- But if the problem asks for the actual path, DSU is the wrong tool



- 1 Merge-only components
- 2 Parent forest representation
- 3 Making DSU fast
- 4 Compact one-array DSU
- 5 Component metadata
- 6 Cycle detection
- 7 DSU vs DFS/BFS
- 8 Rollback DSU**
- 9 Problem patterns

Rollback DSU

Why rollback DSU?

- Normal DSU is destructive: after `unite(a, b)`, the old state is gone



Why rollback DSU?

- Normal DSU is destructive: after `unite(a, b)`, the old state is gone
- Some offline algorithms need to try edges, answer queries, then undo them



Why rollback DSU?

- Normal DSU is destructive: after `unite(a, b)`, the old state is gone
- Some offline algorithms need to try edges, answer queries, then undo them
- Rollback DSU stores enough history to return to an earlier snapshot



Why rollback DSU?

- Normal DSU is destructive: after `unite(a, b)`, the old state is gone
- Some offline algorithms need to try edges, answer queries, then undo them
- Rollback DSU stores enough history to return to an earlier snapshot
- This is useful for offline dynamic connectivity and divide-and-conquer on time



Key tradeoff

- We keep union by size

Key tradeoff

- We keep union by size
- We do **not** use path compression



Key tradeoff

- We keep union by size
- We do **not** use path compression
- Why? Path compression changes many parents during find



Key tradeoff

- We keep union by size
- We do **not** use path compression
- Why? Path compression changes many parents during find
- Those hidden changes are expensive to record and undo



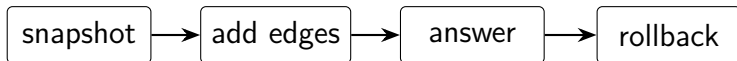
Key tradeoff

- We keep union by size
- We do **not** use path compression
- Why? Path compression changes many parents during find
- Those hidden changes are expensive to record and undo
- Without path compression, rollback DSU is usually $\mathcal{O}(\log n)$ per operation



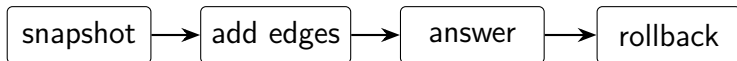
Snapshot idea

- Before a temporary block of work, save `snapshot()`



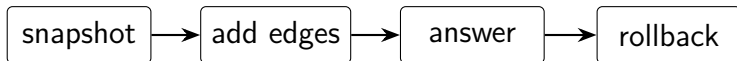
Snapshot idea

- Before a temporary block of work, save `snapshot()`
- Each successful merge records what changed



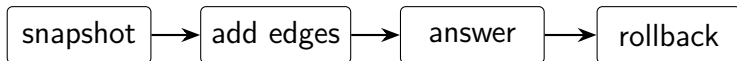
Snapshot idea

- Before a temporary block of work, save `snapshot()`
- Each successful merge records what changed
- After the block, call `rollback(snapshot)`



Snapshot idea

- Before a temporary block of work, save `snapshot()`
- Each successful merge records what changed
- After the block, call `rollback(snapshot)`
- The DSU returns to exactly the old component structure



Rollback DSU state

```
1  struct RollbackDSU {
2      vector<int> e;
3      vector<pair<int,int>> hist;
4
5      RollbackDSU(int n) : e(n, -1) {}
6
7      int get(int x) {
8          while (e[x] >= 0) x = e[x];
9          return x;
10     }
11
12     int snapshot() { return hist.size(); }
13 };
```



Rollback merge and undo

```
1  // methods inside RollbackDSU
2  bool unite(int a, int b) {
3      a = get(a), b = get(b);
4      if (a == b) return false;
5      if (e[a] > e[b]) swap(a, b);
6      hist.push_back({b, e[b]});
7      hist.push_back({a, e[a]});
8      e[a] += e[b];
9      e[b] = a;
10     return true;
11 }
12
13 void rollback(int snap) {
14     while ((int)hist.size() > snap) {
15         auto [v, old] = hist.back();
16         hist.pop_back();
17         e[v] = old;
18     }
19 }
```



When rollback appears

- The graph changes over time, but the whole list of operations is known

When rollback appears

- The graph changes over time, but the whole list of operations is known
- Add each edge to the time intervals where it is alive

When rollback appears

- The graph changes over time, but the whole list of operations is known
- Add each edge to the time intervals where it is alive
- DFS over a segment tree of time intervals

When rollback appears

- The graph changes over time, but the whole list of operations is known
- Add each edge to the time intervals where it is alive
- DFS over a segment tree of time intervals
- At each node: snapshot, add active edges, recurse, then rollback

When rollback appears

- The graph changes over time, but the whole list of operations is known
- Add each edge to the time intervals where it is alive
- DFS over a segment tree of time intervals
- At each node: snapshot, add active edges, recurse, then rollback
- This handles deletions offline without needing normal DSU to split components



- 1 Merge-only components
- 2 Parent forest representation
- 3 Making DSU fast
- 4 Compact one-array DSU
- 5 Component metadata
- 6 Cycle detection
- 7 DSU vs DFS/BFS
- 8 Rollback DSU
- 9 Problem patterns

Problem patterns

Online connectivity

- Process operations in the order they arrive

Online connectivity

- Process operations in the order they arrive
- Edge addition: `unite(a, b)`

Online connectivity

- Process operations in the order they arrive
- Edge addition: `unite(a, b)`
- Connectivity query: check `find(a) == find(b)`



Online connectivity

- Process operations in the order they arrive
- Edge addition: `unite(a, b)`
- Connectivity query: check `find(a) == find(b)`
- This works when updates only add connections



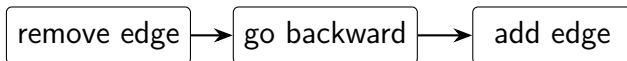
Connectivity query code

```
1 DSU dsu(n);
2
3 for (auto [type, a, b] : queries) {
4     if (type == "add") {
5         dsu.unite(a, b);
6     } else if (type == "ask") {
7         cout << (dsu.same(a, b) ? "YES" : "NO") << '\n';
8     }
9 }
```



Offline reverse trick

- Standard DSU can add edges, but it cannot delete edges
- If all deletions are known first, process time backward
- A deletion forward becomes an addition backward
- Answer each query in reverse order, then reverse the answers at the end



Reverse deletion code

```
1 // Start from the graph after all deletions happened.
2 DSU dsu(n);
3 for (auto [u, v] : edges_still_alive) {
4     dsu.unite(u, v);
5 }
6
7 vector<int> ans;
8 for (int i = q - 1; i >= 0; i--) {
9     if (queries[i].type == "ask") {
10         ans.push_back(dsu.same(queries[i].a, queries[i].b));
11     } else if (queries[i].type == "delete") {
12         // In reverse, this edge comes back.
13         dsu.unite(queries[i].a, queries[i].b);
14     }
15 }
16
17 reverse(ans.begin(), ans.end());
```



Example: Road Construction

CSES: Road Construction

Add roads one by one. After each road, print the number of connected components and the size of the largest component.

Example: Road Construction

CSES: Road Construction

Add roads one by one. After each road, print the number of connected components and the size of the largest component.

- DSU tracks each city component

Example: Road Construction

CSES: Road Construction

Add roads one by one. After each road, print the number of connected components and the size of the largest component.

- DSU tracks each city component
- Successful `unite` decreases the component count



Example: Road Construction

CSES: Road Construction

Add roads one by one. After each road, print the number of connected components and the size of the largest component.

- DSU tracks each city component
- Successful `unite` decreases the component count
- The new component size may update the maximum



Road Construction code

```
1 DSU dsu(n);
2 int components = n;
3 int largest = 1;
4
5 for (int i = 0; i < m; i++) {
6     int a, b;
7     cin >> a >> b;
8
9     if (dsu.unite(a, b)) {
10         components--;
11         largest = max(largest, dsu.size(a));
12     }
13
14     cout << components << ' ' << largest << '\n';
15 }
```



Kruskal preview

- Sort edges by increasing weight

Kruskal preview

- Sort edges by increasing weight
- For each edge (u, v) , ask whether u and v are already connected



Kruskal preview

- Sort edges by increasing weight
- For each edge (u, v) , ask whether u and v are already connected
- If not, add the edge and merge the components

Kruskal preview

- Sort edges by increasing weight
- For each edge (u, v) , ask whether u and v are already connected
- If not, add the edge and merge the components
- DSU makes the cycle check fast



What DSU does not handle

- Standard DSU cannot split a component

What DSU does not handle

- Standard DSU cannot split a component
- Edge deletion is harder than edge addition



What DSU does not handle

- Standard DSU cannot split a component
- Edge deletion is harder than edge addition
- If deletions appear, look for an offline reverse process or a more advanced data structure



What DSU does not handle

- Standard DSU cannot split a component
- Edge deletion is harder than edge addition
- If deletions appear, look for an offline reverse process or a more advanced data structure
- DSU answers component-level questions, not path-level questions



Common mistakes

- Forgetting to initialize every `parent[i] = i`
- Updating `sz` on a non-root vertex
- Returning `true` from `unite` when both vertices were already connected
- Mixing 0-indexed and 1-indexed vertices
- Assuming DSU can remove edges



Practice problems

- CSES: Road Construction – dynamic component count and largest size
- CSES: Building Roads – connect all components with extra roads
- CSES: Road Reparation – Kruskal's MST algorithm
- CSES: Network Breakdown – process deletions offline in reverse



The End

Thanks for listening.

© 2026 Sean. Please do not repost or redistribute without permission.

